Exp. No.: 6	Error Correction at Data Link Layer (Hamming Code)

Name: Mirudhula K | Roll No: 241501109

Aim

To write a program to implement error detection and correction using the Hamming code concept, and to make a test run to input a data stream and verify the error correction feature.

Required Software / Tools

- Python 3 (or any general-purpose programming language)

Theory / Concepts

Hamming code is a set of error-correction codes that can be used to detect and correct errors that occur when data is transmitted from a sender to a receiver. It is a technique developed by R. W. Hamming for error correction. Redundant (parity) bits are inserted into the data at specific positions so that, on the receiving end, any single-bit error can be located and corrected by checking the parity bits.

Procedure

Sender program:

1. Take a text input of any length and convert it to binary.
2. Apply the Hamming code concept on the binary data (encoding each 4-bit block into a 7-bit Hamming codeword) to add redundant bits.
3. Save this encoded output in a file called "channel.txt".

Receiver program:

1. Read the input from the channel file.
2. Apply the Hamming code check on the binary data to check for errors.
3. If there is an error, display the position of the error.
4. Otherwise, remove the redundant bits, convert the binary data to ASCII, and display the output.

Program / Code

Sender Program (sender.py)

```
# Hamming Code Sender Program
# Converts input text to binary, applies Hamming(7,4) code to each 4-bit
# nibble (adding 3 redundant/parity bits), and writes the encoded bit
# stream to a file called "channel.txt".

def text_to_bits(text):
    return ''.join(format(ord(ch), '08b') for ch in text)

def hamming_encode(d):
    # d = [d1, d2, d3, d4]  (4 data bits)
    d1, d2, d3, d4 = d
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4
    # Codeword positions: p1 p2 d1 p4 d2 d3 d4
    return [p1, p2, d1, p4, d2, d3, d4]

def encode_stream(bitstring):
    # Pad to a multiple of 4 bits
```

```
    while len(bitstring) % 4 != 0:
        bitstring += '0'
    encoded = ''
    for i in range(0, len(bitstring), 4):
        nibble = [int(b) for b in bitstring[i:i+4]]
        codeword = hamming_encode(nibble)
        encoded += ''.join(str(b) for b in codeword)
    return encoded

if __name__ == "__main__":
    text = input("Enter text to send: ")
    bits = text_to_bits(text)
    print("Binary form of input :", bits)

    encoded_bits = encode_stream(bits)
    print("Hamming encoded data :", encoded_bits)

    with open("channel.txt", "w") as f:
        f.write(encoded_bits)
    print("Encoded data written to channel.txt")
```

Receiver Program (receiver.py)

```
# Hamming Code Receiver Program
# Reads the encoded bit stream from "channel.txt", applies Hamming(7,4)
# error checking on each 7-bit codeword, reports the position of any
# single-bit error found, corrects it, then reconstructs and displays
# the original ASCII text.

def hamming_decode(codeword):
    # codeword = [p1, p2, d1, p4, d2, d3, d4]  (positions 1..7)
    c = codeword[:]
    p1, p2, d1, p4, d2, d3, d4 = c

    c1 = p1 ^ d1 ^ d2 ^ d4
    c2 = p2 ^ d1 ^ d3 ^ d4
    c4 = p4 ^ d2 ^ d3 ^ d4

    error_pos = c1 * 1 + c2 * 2 + c4 * 4   # 0 = no error

    if error_pos != 0:
        # Flip the erroneous bit (1-indexed position) to correct it
        c[error_pos - 1] ^= 1

    p1, p2, d1, p4, d2, d3, d4 = c
    data_bits = [d1, d2, d3, d4]
    return data_bits, error_pos

def bits_to_text(bitstring):
    chars = []
    for i in range(0, len(bitstring), 8):
        byte = bitstring[i:i+8]
        if len(byte) == 8:
            chars.append(chr(int(byte, 2)))
    return ''.join(chars)

if __name__ == "__main__":
    with open("channel.txt", "r") as f:
        received = f.read().strip()

    print("Received data from channel:", received)

    decoded_bits = ''
```

```
    for i in range(0, len(received), 7):
        codeword = [int(b) for b in received[i:i+7]]
        if len(codeword) < 7:
            continue
        data_bits, error_pos = hamming_decode(codeword)
        if error_pos != 0:
            print(f"Error detected in codeword {i // 7 + 1} at bit position
{error_pos} -- corrected.")
        decoded_bits += ''.join(str(b) for b in data_bits)

    # Remove padding added by the sender, then convert back to text
    original_bits = decoded_bits[:len(decoded_bits) - (len(decoded_bits) % 8)]
    text = bits_to_text(original_bits)
    print("Recovered original message:", text)
```

Input

Sample input text given to the sender program:

```
Enter text to send: HI
```

Output

Run 1 — Channel data received without any error:

```
Binary form of input : 0100100001001001
Hamming encoded data : 100110011100001001100001100l001
Encoded data written to channel.txt

Received data from channel: 100110011100001001100001100l001
Recovered original message: HI
```

Run 2 — A single bit was flipped in channel.txt to simulate a transmission error:

```
Corrupted channel data: 101110011100001001100001100l001

Received data from channel: 101110011100001001100001100l001
Error detected in codeword 1 at bit position 3 -- corrected.
Recovered original message: HI
```

Result

A sender-receiver program pair implementing the Hamming code concept was successfully developed and tested. The receiver correctly recovered the original message when no error was present, and correctly detected and corrected a single-bit error introduced into the channel data, verifying the error correction feature of the Hamming code.